

Mining_HW1

2025-02-28

#Problem 1

Given, training MSE is $R_{tr}(\hat{\beta}) = \frac{1}{N} \sum_{i=1}^N (y_i - x_i^T \hat{\beta})^2$ and test MSE is $R_{te}(\hat{\beta}) = \frac{1}{M} \sum_{i=1}^M (\tilde{y}_i - \tilde{x}_i^T \hat{\beta})^2$

We need to show that, $E[R_{tr}(\hat{\beta})] \leq E[R_{te}(\hat{\beta})]$

The $\hat{\beta}$ here is the least square estimates from the training data which is chosen to minimize the square residuals (training error). It fits better for training data. But, we are using the same $\hat{\beta}$ in calculating test MSE, with data that is unknown this will lead to larger MSE than the training MSE or equal.

Hence, $E[R_{tr}(\hat{\beta})] \leq E[R_{te}(\hat{\beta})]$

#Problem 2

Method 1 to construct confidence sets for β (From Lecture 1, slide 36-42)

Given, $y_i = \beta_0 + x_i^T \beta + \epsilon_i$, $\epsilon_i \sim N(0, \sigma^2)$

Least square estimator is $\hat{\beta} = (X^T X)^{-1} X^T y$ $SE(\hat{\beta}) = \sqrt{\sigma^2 (X^T X)^{-1}}$ Estimate of Variance σ^2 is $\hat{\sigma}^2 = \frac{1}{N-p-1} \sum_{i=1}^N (y_i - \hat{y}_i)^2$ where $\hat{y} = X \hat{\beta}$ $t_j = \frac{\hat{\beta}_j - \beta_j}{SE(\hat{\beta}_j)}$ when the sample size N is large, the distribution of the estimated coefficients is approximately normal $\frac{\hat{\beta}_j - \beta_j}{\hat{\sigma} \sqrt{v_j}} \sim N(0, 1)$ Using this, a $((1-\alpha) \times 100\%)$ confidence interval for each component (β_j) is: $(\hat{\beta}_j - z^{(1-\alpha/2)} \hat{\sigma} \sqrt{v_j}, \hat{\beta}_j + z^{(1-\alpha/2)} \hat{\sigma} \sqrt{v_j})$ where, $(z^{(1-\alpha/2)})$ is the $((1-\alpha/2))$ th percentile of the standard normal distribution. If (N) is not large, we replace it with the (t)-distribution percentile $(t_{N-p-1}^{(1-\alpha/2)})$.

#Problem 3

First we start with, Given,

$$\begin{aligned}\bar{x}_j &= \frac{1}{N} \sum_{i=1}^N x_{ij} \\ x_{ij} &= (x_{ij} - \bar{x}_j) + \bar{x}_j \\ y_i &= \beta_0 + \sum_{j=1}^p x_{ij} \beta_j + \epsilon_i \\ y_i &= \beta_0 + \sum_{j=1}^p ((x_{ij} - \bar{x}_j) + \bar{x}_j) \beta_j + \epsilon_i \\ y_i &= \beta_0 + \sum_{j=1}^p (x_{ij} - \bar{x}_j) \beta_j + \sum_{j=1}^p \bar{x}_j \beta_j + \epsilon_i \\ y_i - \sum_{j=1}^p \bar{x}_j \beta_j &= \beta_0 + \sum_{j=1}^p (x_{ij} - \bar{x}_j) \beta_j + \epsilon_i \\ \beta_0^c &= \beta_0 + \sum_{j=1}^p \bar{x}_j \beta_j\end{aligned}$$

Since the predictors are now centered, say

$$\beta_j^c = \beta_j$$

Thus,

$$y_i - \sum_{j=1}^p \bar{x}_j \beta_j = \beta_0^c + \sum_{j=1}^p (x_{ij} - \bar{x}_j) \beta_j^c + \epsilon_i$$

Now, we can say that

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \left\{ \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}$$

is equivalent to

$$\hat{\beta}^c = \arg \min_{\beta^c \in \mathbb{R}^{p+1}} \left\{ \sum_{i=1}^N \left(y_i - \beta_0^c - \sum_{j=1}^p (x_{ij} - \bar{x}_j) \beta_j^c \right)^2 + \lambda \sum_{j=1}^p (\beta_j^c)^2 \right\}$$

Similarly for lasso, First we start with,

Given,

$$\bar{x}_j = \frac{1}{N} \sum_{i=1}^N x_{ij}$$

$$x_{ij} = (x_{ij} - \bar{x}_j) + \bar{x}_j$$

$$y_i = \beta_0 + \sum_{j=1}^p x_{ij} \beta_j + \epsilon_i$$

$$y_i = \beta_0 + \sum_{j=1}^p ((x_{ij} - \bar{x}_j) + \bar{x}_j) \beta_j + \epsilon_i$$

$$y_i = \beta_0 + \sum_{j=1}^p (x_{ij} - \bar{x}_j) \beta_j + \sum_{j=1}^p \bar{x}_j \beta_j + \epsilon_i$$

$$y_i - \sum_{j=1}^p \bar{x}_j \beta_j = \beta_0 + \sum_{j=1}^p (x_{ij} - \bar{x}_j) \beta_j + \epsilon_i$$

$$\beta_0^c = \beta_0 + \sum_{j=1}^p \bar{x}_j \beta_j$$

Since the predictors are now centered, say

$$\beta_j^c = \beta_j$$

Thus,

$$y_i - \sum_{j=1}^p \bar{x}_j \beta_j = \beta_0^c + \sum_{j=1}^p (x_{ij} - \bar{x}_j) \beta_j^c + \epsilon_i$$

Now, we can say that

$$\hat{\beta}^{\text{lasso}} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \left\{ \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

is equivalent to

$$\hat{\beta}^c = \arg \min_{\beta^c \in \mathbb{R}^{p+1}} \left\{ \sum_{i=1}^N \left(y_i - \beta_0^c - \sum_{j=1}^p (x_{ij} - \bar{x}_j) \beta_j^c \right)^2 + \lambda \sum_{j=1}^p |\beta_j^c| \right\}$$

#Problem 4 Part B

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
set.seed(123)
```

```
n <- 1000
```

```
p <- 10000
```

```
beta <- rep(1, p)
```

```
X <- matrix(sample(c(-1, 1), n * p, replace = TRUE), n, p)
```

```
error <- rnorm(n, mean = 0, sd = 1)
```

```
y <- X %*% beta + error
```

```
lambda_values <- c(0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1.0, 2.0, 5.0, 10, 20, 50, 100)
```

```
cv_ridge <- cv.glmnet(X, y, alpha = 0, lambda = lambda_values, nfolds = 10)
```

```
best_lambda <- cv_ridge$lambda.min
```

```
ridge_model <- glmnet(X, y, alpha = 0, lambda = best_lambda)
```

```
predicted_y <- predict(ridge_model, X)
```

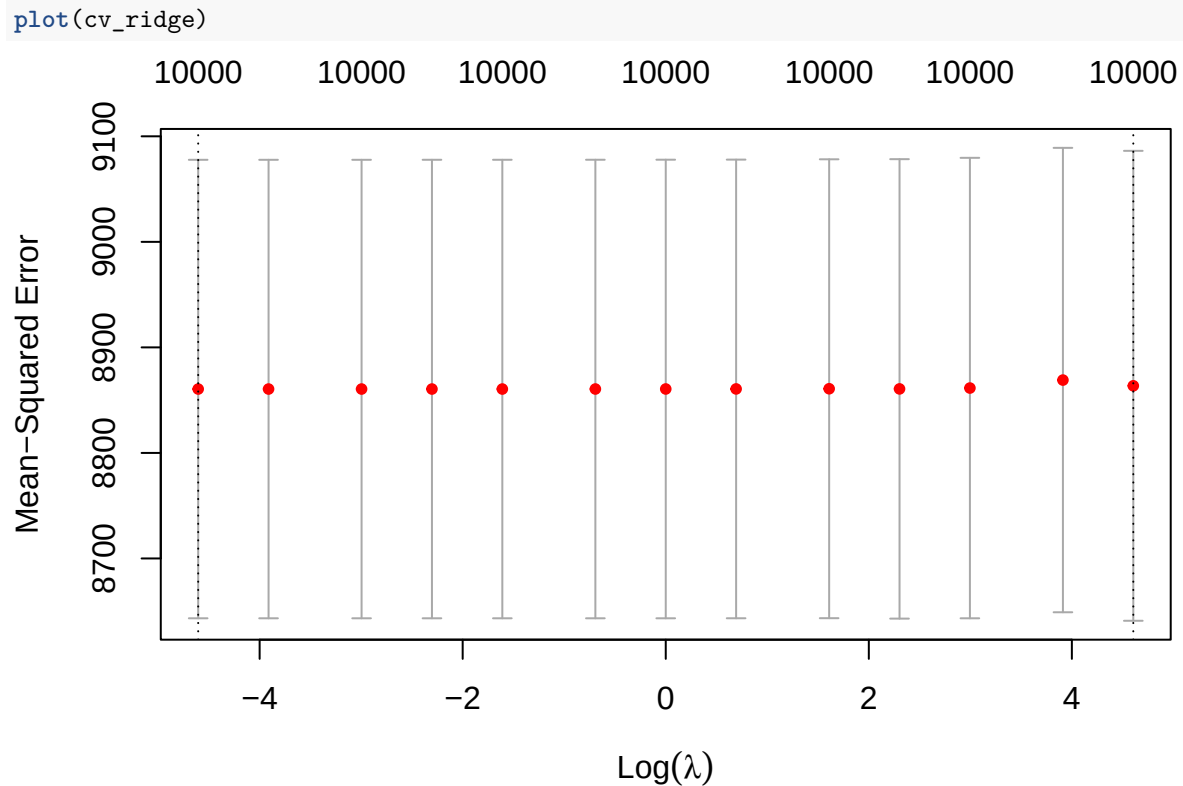
```
risk_f <- mean((predicted_y - y)^2)
```

```
print(paste("Optimal lambda:", best_lambda))
```

```
## [1] "Optimal lambda: 0.01"
```

```
print(paste("Corresponding risk:", risk_f))
```

```
## [1] "Corresponding risk: 3.62055060944893e-08"
```



#Problem 5 (Problem 8, Page 262)

```
#install.packages("leaps")
library(leaps)
library(glmnet)
library(ggplot2)
```

Part A

```
set.seed(123)
n <- 100
X <- rnorm(n)
noise <- rnorm(n)
head(X)
```

```
## [1] -0.56047565 -0.23017749 1.55870831 0.07050839 0.12928774 1.71506499
```

```
head(noise)
```

```
## [1] -0.71040656 0.25688371 -0.24669188 -0.34754260 -0.95161857 -0.04502772
```

Part B

```
beta_0 <- 2
beta_1 <- 3
beta_2 <- -2
beta_3 <- 1
Y <- beta_0 + beta_1 * X + beta_2 * X^2 + beta_3 * X^3 + noise
head(Y)
```

```
## [1] -1.196163 1.448193 5.357283 1.854390 1.404975 6.262046
```

```
data_set <- data.frame(Y=Y, X=X)
```

```
for (i in 2:10) {
  data_set[[paste0("X", i)]] <- X^i
}
```

```
head(data_set)
```

```
##           Y           X           X2           X3           X4           X5
## 1 -1.196163 -0.56047565 0.314132950 -0.1760638685 9.867951e-02 -5.530746e-02
## 2  1.448193 -0.23017749 0.052981677 -0.0121951893 2.807058e-03 -6.461216e-04
## 3  5.357283  1.55870831 2.429571609  3.7869934661 5.902818e+00  9.200772e+00
## 4  1.854390  0.07050839 0.004971433  0.0003505278 2.471515e-05  1.742625e-06
## 5  1.404975  0.12928774 0.016715318  0.0021610857 2.794019e-04  3.612324e-05
## 6  6.262046  1.71506499 2.941447909  5.0447743199 8.652116e+00  1.483894e+01
##           X6           X7           X8           X9           X10
## 1 3.099849e-02 -1.737390e-02 9.737646e-03 -5.457713e-03 3.058915e-03
## 2 1.487226e-04 -3.423260e-05 7.879575e-06 -1.813701e-06 4.174731e-07
## 3 1.434132e+01  2.235393e+01 3.484326e+01  5.431048e+01 8.465420e+01
## 4 1.228697e-07  8.663346e-09 6.108386e-10  4.306925e-11 3.036743e-12
## 5 4.670291e-06  6.038114e-07 7.806541e-08  1.009290e-08 1.304888e-09
## 6 2.544975e+01  4.364797e+01 7.485911e+01  1.283882e+02 2.201942e+02
```

Part C

```
best_subset <- regsubsets(Y ~ ., data = data_set, nvmax = 10)
```

```
subset_summary <-summary(best_subset)
```

```
summary(best_subset)
```

```
## Subset selection object
```

```
## Call: regsubsets.formula(Y ~ ., data = data_set, nvmax = 10)
```

```
## 10 Variables (and intercept)
```

```
##      Forced in Forced out
```

```
## X      FALSE      FALSE
```

```
## X2     FALSE     FALSE
```

```
## X3     FALSE     FALSE
```

```
## X4     FALSE     FALSE
```

```
## X5     FALSE     FALSE
```

```
## X6     FALSE     FALSE
```

```
## X7     FALSE     FALSE
```

```
## X8     FALSE     FALSE
```

```
## X9     FALSE     FALSE
```

```
## X10    FALSE     FALSE
```

```
## 1 subsets of each size up to 10
```

```
## Selection Algorithm: exhaustive
```

```
##           X    X2  X3  X4  X5  X6  X7  X8  X9  X10
```

```
## 1  ( 1 )  "*"  " "  " "  " "  " "  " "  " "  " "  " "
```

```
## 2  ( 1 )  " "  "*"  "*"  " "  " "  " "  " "  " "  " "
```

```
## 3  ( 1 )  "*"  "*"  "*"  " "  " "  " "  " "  " "  " "
```

```
## 4  ( 1 )  "*"  "*"  "*"  " "  " "  " "  "*"  " "  " "
```

```
## 5  ( 1 )  "*"  "*"  "*"  " "  " "  " "  "*"  " "  "*"  "
```

```
## 6  ( 1 )  "*"  " "  "*"  "*"  " "  " "  "*"  " "  "*"  "
```

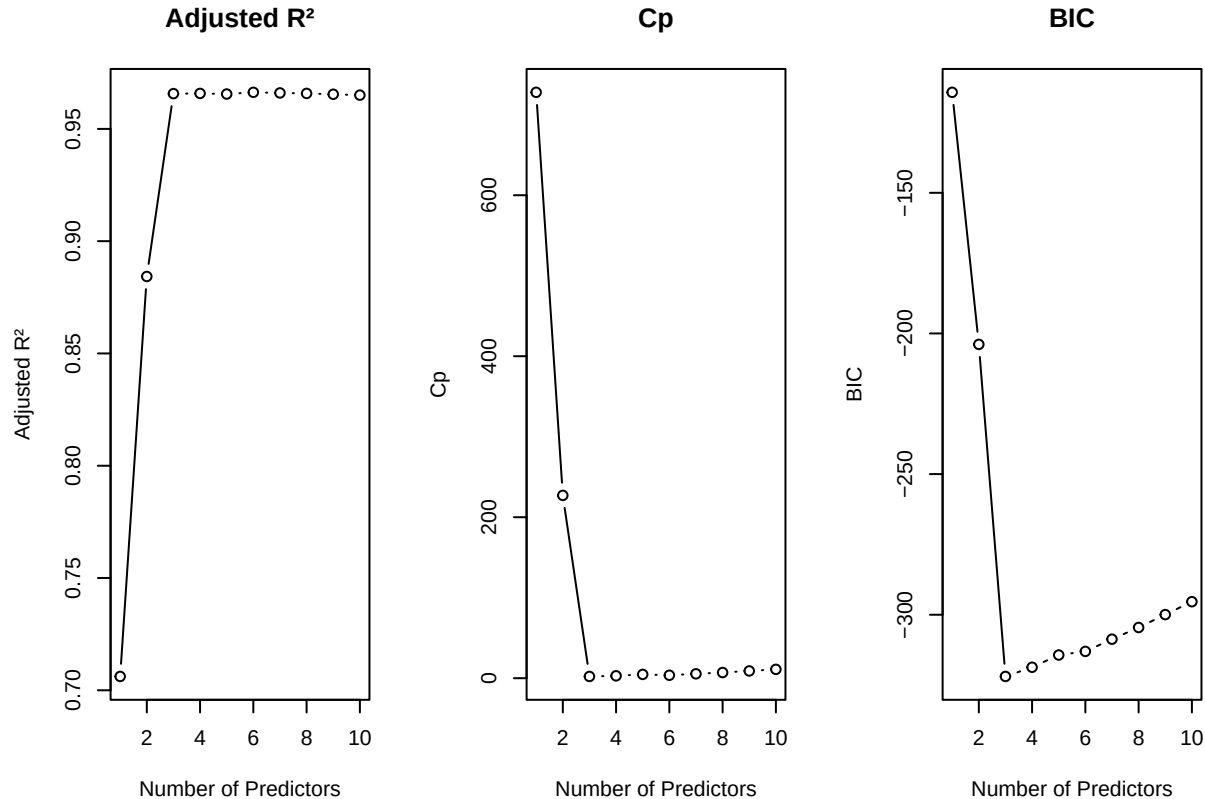
```
## 7  ( 1 )  "*"  " "  " "  " "  "*"  "*"  "*"  "*"  " "  "*"  "
```

```
## 8  ( 1 )  "*"  " "  " "  "*"  "*"  "*"  "*"  "*"  " "  "*"  "
```

```
## 9  ( 1 )  "*"  " "  " "  "*"  "*"  "*"  "*"  "*"  "*"  "*"  "
```

```
## 10 ( 1 ) "*" "*" "*" "*" "*" "*" "*" "*" "*" "*"
```

```
par(mfrow = c(1, 3))
plot(subset_summary$adjr2, type = "b", xlab = "Number of Predictors", ylab = "Adjusted R2", main = "Adjusted R2")
plot(subset_summary$cp, type = "b", xlab = "Number of Predictors", ylab = "Cp", main = "Cp")
plot(subset_summary$bic, type = "b", xlab = "Number of Predictors", ylab = "BIC", main = "BIC")
```



```
print(paste("Best model size according to BIC:", which.min(subset_summary$bic)))
```

```
## [1] "Best model size according to BIC: 3"
```

```
print(paste("Best model size according to Cp:", which.min(subset_summary$cp)))
```

```
## [1] "Best model size according to Cp: 3"
```

```
print(paste("Best model size according to Adjusted R2:", which.max(subset_summary$adjr2)))
```

```
## [1] "Best model size according to Adjusted R2: 6"
```

As BIC and Adjusted C_p has best size 3, we use that. For these, the plot becomes almost constant after 3 predictors but for BIC it increases after 3. We can see from BIC, that after 3, adding more variable will make model too complex and from adjusted R^2 and C_p we can see that after 3, there is no improvement by adding more variables. So, 3 would be the best it might be a little complex but will have more accuracy.

Part D

```
best_subset_f <- regsubsets(Y ~ ., data = data_set, method = "forward", nvmax = 10)
best_subset_b <- regsubsets(Y ~ ., data = data_set, method = "backward", nvmax = 10)
forward_summary <- summary(best_subset_b)
backward_summary <- summary(best_subset_f)
```

```
best_forward <- which.min(forward_summary$bic)
best_backward <- which.min(backward_summary$bic)
```

```
print("Best model coefficients (Forward Stepwise Selection):")
```

```
## [1] "Best model coefficients (Forward Stepwise Selection):"
```

```
coef(best_subset_f, best_forward)
```

```
## (Intercept)          X          X2          X3          X5          X6
## 2.035201151  3.112343440 -2.228539342  0.640038876  0.145500030  0.005910910
##          X9
## -0.003124974
```

```
print("Best model coefficients (Backward Stepwise Selection):")
```

```
## [1] "Best model coefficients (Backward Stepwise Selection):"
```

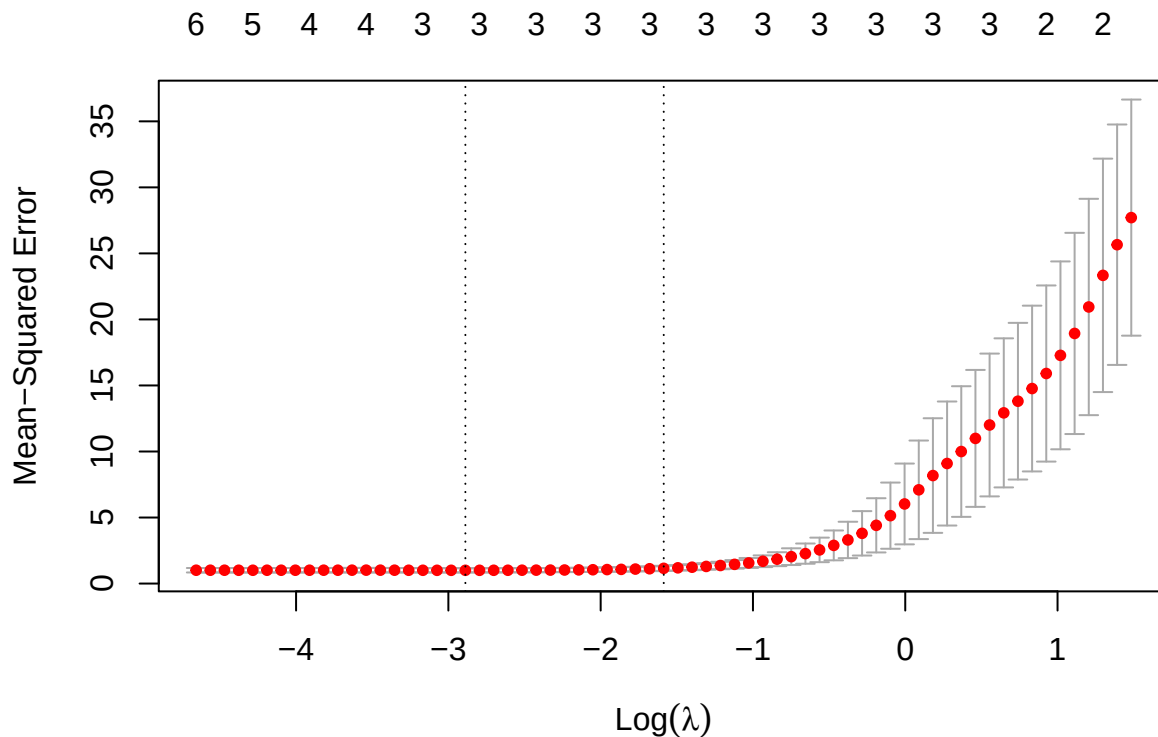
```
coef(best_subset_f, best_backward)
```

```
## (Intercept)          X          X2          X3
## 1.970394      2.920446  -2.091543  1.020436
```

Forward selection chose 7 predictors , but backward selection chose 4 predictors. It omitted X5,X6 and X9.

#Part E

```
X_matrix <- as.matrix(data_set[, -1])
lasso <- cv.glmnet(X_matrix, Y, alpha = 1, nfolds = 10)
best_lambda <- lasso$lambda.min
lasso_model <- glmnet(X_matrix, Y, alpha = 1, lambda = best_lambda)
lasso_coef <- coef(lasso_model)
plot(lasso)
```



```
print(paste("Optimal lambda (Lasso):", best_lambda))
```

```
## [1] "Optimal lambda (Lasso): 0.0556677959516223"
```

```
print("Lasso coefficients:")
```

```
## [1] "Lasso coefficients:"
```

```
print(lasso_coef)
```

```
## 11 x 1 sparse Matrix of class "dgCMatrix"
```

```
##              s0
```

```
## (Intercept)  1.926284
```

```
## X           2.884494
```

```
## X2          -2.029350
```

```
## X3           1.003936
```

```
## X4           .
```

```
## X5           .
```

```
## X6           .
```

```
## X7           .
```

```
## X8           .
```

```
## X9           .
```

```
## X10          .
```

Lasso selects slightly different set of variables.

Part F

```
set.seed(123)
```

```
n <- 100
```

```
X <- rnorm(n)
```

```
noise <- rnorm(n)
```

```
beta_0 <- 2
```

```
beta_7 <- 2
```

```
Y_new <- beta_0 + beta_7 * X^7 + noise
```

```
dataset_new <- data.frame(Y = Y_new, X)
```

```
for (i in 2:10) {
```

```
  dataset_new[[paste0("X", i)]] <- X^i
```

```
}
```

```
#print(head(data_new))
```

```
X_matrix_new <- as.matrix(dataset_new[, -1, drop = FALSE])
```

```
Y_new <- dataset_new$Y
```

```
best_subset_new <- regsubsets(Y ~ ., data = dataset_new, nvmax = 10)
```

```
summary(best_subset_new)
```

```
## Subset selection object
```

```
## Call: regsubsets.formula(Y ~ ., data = dataset_new, nvmax = 10)
```

```
## 10 Variables (and intercept)
```

```
##      Forced in Forced out
```

```
## X      FALSE      FALSE
```

```
## X2     FALSE     FALSE
```

```
## X3     FALSE     FALSE
```

```
## X4     FALSE     FALSE
```

```
## X5     FALSE     FALSE
```

```
## X6     FALSE     FALSE
```

```
## X7     FALSE     FALSE
```

```
## X8     FALSE     FALSE
```

```
## X9     FALSE     FALSE
```

```
## X10    FALSE     FALSE
```

```
## 1 subsets of each size up to 10
```



```

## Selection Algorithm: exhaustive
##      X   X2  X3  X4  X5  X6  X7  X8  X9  X10
## 1 ( 1 ) " " " " " " " " " " " " "*" " " " " " "
## 2 ( 1 ) " " "*" " " " " " " " " "*" " " " " " "
## 3 ( 1 ) " " "*" " " " " " " " "*" "*" " " " " " "
## 4 ( 1 ) " " " " " " "*" " " " "*" "*" "*" " " " "
## 5 ( 1 ) " " "*" " " " "*" " " " "*" "*" "*" " " "
## 6 ( 1 ) " " "*" " " " "*" " " " "*" "*" "*" " " "*"
## 7 ( 1 ) "*" "*" " " " "*" " " " "*" "*" "*" " " "*"
## 8 ( 1 ) " " "*" "*" "*" " " " "*" "*" "*" "*" "*"
## 9 ( 1 ) "*" "*" "*" "*" "*" "*" "*" "*" " " " "*"
## 10 ( 1 ) "*" "*" "*" "*" "*" "*" "*" "*" "*" "*"

lasso_new <- cv.glmnet(X_matrix_new, Y_new, alpha = 1, nfolds = 10)
best_lambda_new <- lasso_new$lambda.min
lasso_model_new <- glmnet(X_matrix_new, Y_new, alpha = 1, lambda = best_lambda_new)
lasso_coef_new <- coef(lasso_model_new)
print(paste("Optimal lambda for new model:", best_lambda_new))

## [1] "Optimal lambda for new model: 3.0576123170359"

print("Lasso coefficients for new model:")

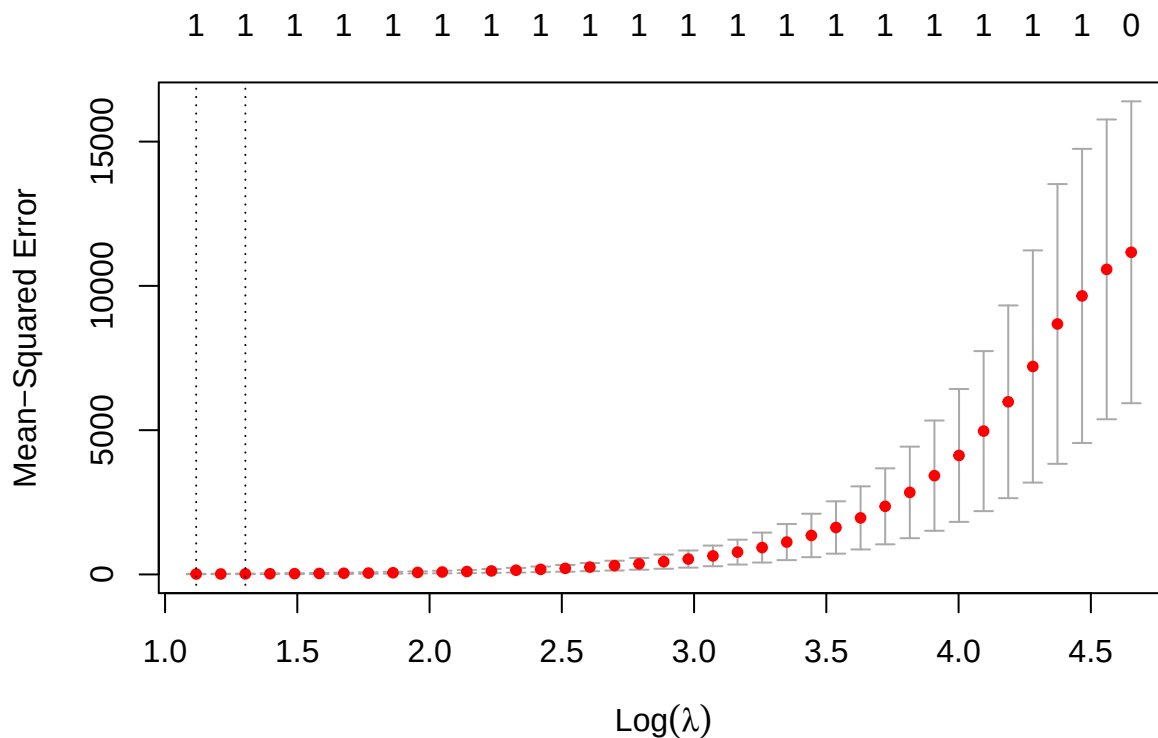
## [1] "Lasso coefficients for new model:"

print(lasso_coef_new)

## 11 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept) 2.0535998863
## X              .
## X2             .
## X3             .
## X4             .
## X5             .
## X6             .
## X7            1.9390812082
## X8             .
## X9            0.0003877265
## X10            .

plot(lasso_new)

```



#Problem 6 (Problem 9, Page 263)

```
#install.packages("ISLR2")
#install.packages("pls")
#install.packages("caret")
library(ISLR2)
library(glmnet)
```

Part A

```
data(College)
#print(College,n=10)

set.seed(123)
train_size <- sample(1:nrow(College), size = 0.7 * nrow(College))
train_data <- College[train_size, ]
test_data <- College[-train_size, ]

train_X <- model.matrix(Apps ~ ., data = train_data)[, -1]
train_Y <- train_data$Apps

test_X <- model.matrix(Apps ~ ., data = test_data)[, -1]
test_Y <- test_data$Apps
```

Part B

```
model <- lm(Apps ~ ., data = train_data)
model_pred <- predict(model, newdata = test_data)

test_mse <- mean((model_pred - test_Y)^2)

print(paste("Test Error:", test_mse))
```

```
## [1] "Test Error: 1734840.97722585"
```

Part C

```
ridge <- cv.glmnet(train_X, train_Y, alpha = 0, nfolds = 10)
best_lambda_r <- ridge$lambda.min
ridge_model <- glmnet(train_X, train_Y, alpha = 0, lambda = best_lambda)
ridge_pred <- predict(ridge_model, newx = test_X)
ridge_mse <- mean((ridge_pred - test_Y)^2)
print(paste("Ridge Regression Test Error:", ridge_mse))
```

```
## [1] "Ridge Regression Test Error: 1742802.04664401"
```

```
print(paste("Optimal lambda chosen for Ridge by Cross Validation:", best_lambda_r))
```

```
## [1] "Optimal lambda chosen for Ridge by Cross Validation: 314.252370815755"
```

Part D

```
lasso <- cv.glmnet(train_X, train_Y, alpha = 1, nfolds = 10)
best_lambda_l <- lasso$lambda.min
lasso_model <- glmnet(train_X, train_Y, alpha = 1, lambda = best_lambda_l)
lasso_pred <- predict(lasso_model, newx = test_X)
lasso_mse <- mean((lasso_pred - test_Y)^2)
coef <- coef(lasso_model)
num_nonzero <- sum(coef != 0) - 1
print(paste("Lasso Regression Test Error:", lasso_mse))
```

```
## [1] "Lasso Regression Test Error: 1740542.87665424"
```

```
print(paste("Optimal lambda chosen for Lasso by Cross Validation:", best_lambda_l))
```

```
## [1] "Optimal lambda chosen for Lasso by Cross Validation: 8.15492510756652"
```

```
print(paste("Number of nonzero coefficients", num_nonzero))
```

```
## [1] "Number of nonzero coefficients 16"
```

Part E

```
library(pls)
```

```
##
```

```
## Attaching package: 'pls'
```

```
## The following object is masked from 'package:caret':
```

```
##
```

```
##      R2
```

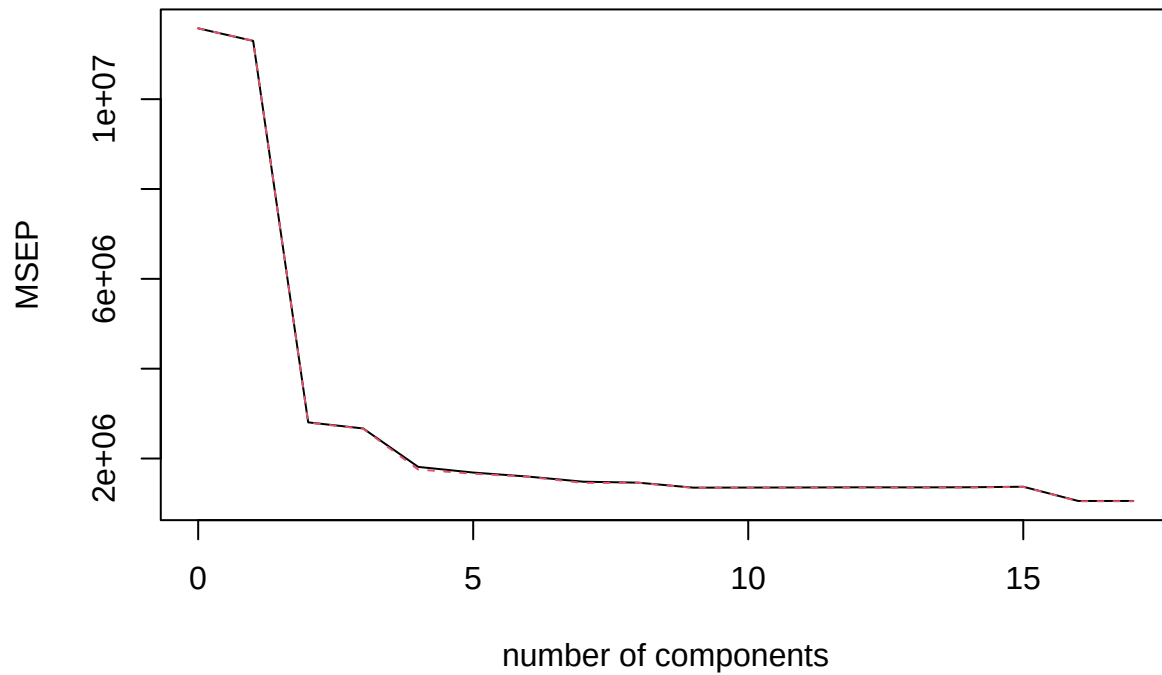
```
## The following object is masked from 'package:stats':
```

```
##
```

```
##      loadings
```

```
pcr_model <- pcr(Apps ~ ., data = train_data, scale = TRUE, validation = "CV")
validationplot(pcr_model, val.type = "MSEP")
```

Apps



```
pcr_pred <- predict(pcr_model, test_data, ncomp = which.min(pcr_model$validation$PRESS))
pcr_mse <- mean((pcr_pred - test_Y)^2)
print(paste("PCR Test Error:", pcr_mse))
```

```
## [1] "PCR Test Error: 1853634.78296277"
```

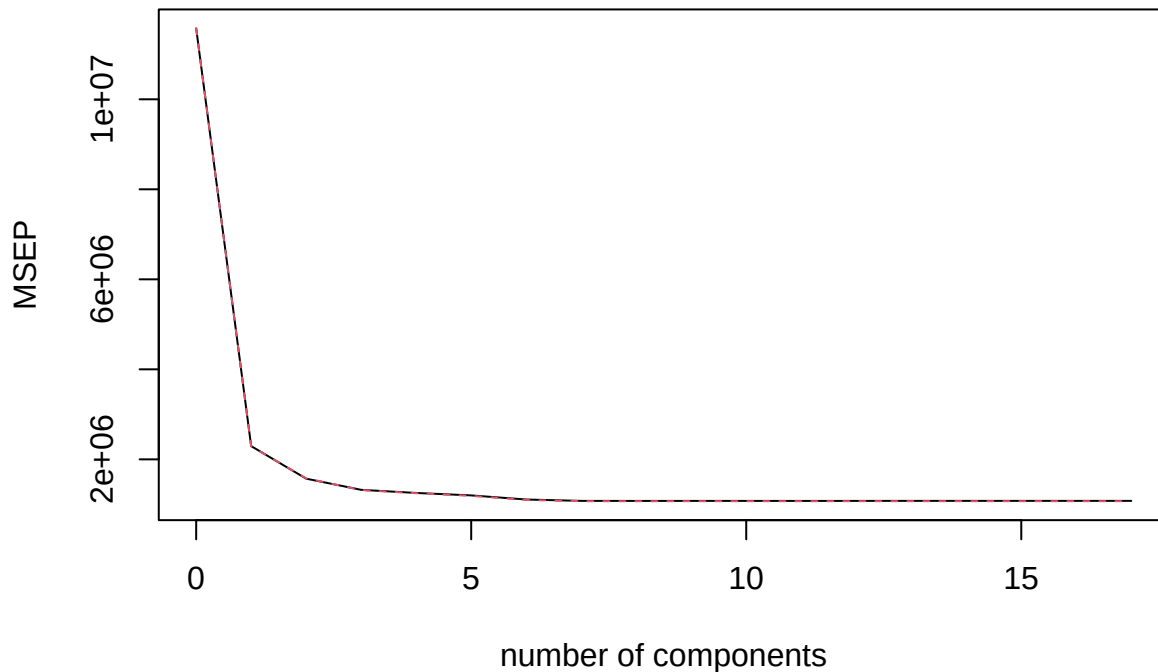
```
print(paste("Optimal number of components (M) for PCR model:", which.min(pcr_model$validation$PRESS)))
```

```
## [1] "Optimal number of components (M) for PCR model: 16"
```

Part F

```
pls_model <- plsr(Apps ~ ., data = train_data, scale = TRUE, validation = "CV")
validationplot(pls_model, val.type = "MSEP")
```

Apps



```
pls_pred <- predict(pls_model, test_data, ncomp = which.min(pls_model$validation$PRESS))
pls_mse <- mean((pls_pred - test_Y)^2)
print(paste("PLS Test Error:", pls_mse))
```

```
## [1] "PLS Test Error: 1774522.41543271"
```

```
print(paste("Optimal number of components (M) for PLS model:", which.min(pls_model$validation$PRESS)))
```

```
## [1] "Optimal number of components (M) for PLS model: 8"
```

Part G

```
print(paste("Linear Regression Test Error:", test_mse))
```

```
## [1] "Linear Regression Test Error: 1734840.97722585"
```

```
print(paste("Ridge Regression Test Error:", ridge_mse))
```

```
## [1] "Ridge Regression Test Error: 1742802.04664401"
```

```
print(paste("Lasso Regression Test Error:", lasso_mse))
```

```
## [1] "Lasso Regression Test Error: 1740542.87665424"
```

```
print(paste("PCR Test Error:", pcr_mse))
```

```
## [1] "PCR Test Error: 1853634.78296277"
```

```
print(paste("PLS Test Error:", pls_mse))
```

```
## [1] "PLS Test Error: 1774522.41543271"
```